

0.8.3. La fonction `nombre()` — écrire un nombre

`#include <nombre>` : `nombre(colonne, ligne, valeur)` écrit la valeur d'une variable en chiffres, sur trois chiffres.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `nombre(colonne, ligne, valeur)` écrit un **nombre calculé** en chiffres : la valeur d'une variable, un calcul. Trois chiffres par défaut : 42 s'écrit **042**.

La ligne à écrire : `#include <nombre>`.

Ses arguments : la colonne, la ligne, la valeur ; un **quatrième**, facultatif, dit combien de chiffres (1, 2 ou 3).

Ce qu'elle coûte : environ **320 octets** la première fois : la routine qui découpe un nombre en chiffres, et les dix chiffres de la police.

Essaie : `nombre(4, 10, score, 2);` : deux chiffres seulement.

On retrouve nombre dans 128 leçons

[0.20](#), [0.20.1](#), [0.20.2](#), [0.23](#), [0.23.1](#), [0.23.2](#), [0.24](#), [0.25](#), [0.26](#), [0.27](#), [0.28](#), [0.29](#), [0.31](#), [0.68](#), [0.68.1](#), [0.68.2](#), [0.68.3](#), [0.69](#), [0.69.1](#), [0.69.2](#), [0.69.3](#), [0.75](#), [0.75.1](#), [0.75.2](#), [0.76](#), [0.76.1](#), [0.76.2](#), [0.76.3](#), [0.76.4](#), [0.76.5](#), [0.76.6](#), [0.76.7](#), [0.76.8](#), [0.76.10](#), [0.76.11](#), [0.76.12](#), [0.76.13](#), [0.76.14](#), [0.79.1](#), [0.80](#), [0.80.1](#), [0.80.2](#), [0.81](#), [0.81.1](#), [0.81.3](#), [0.82.1](#), [0.83](#), [0.83.2](#), [0.84](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [7](#), [9](#), [10](#), [11](#), [12](#), [15](#), [16](#), [18](#), [22](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#), [30](#), [31](#), [32](#), [36](#), [56](#), [59](#), [65](#), [67](#), [68](#), [69](#), [70](#), [71](#), [75](#), [77](#), [78](#), [79](#), [81](#), [82](#), [86](#), [87](#), [88](#), [89](#), [90](#), [91](#), [94](#), [102](#), [111](#), [114](#), [116](#), [117](#), [119](#), [120](#), [122](#), [124](#), [125](#), [129](#), [133](#), [138](#), [139](#), [141](#), [142](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **394 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	53 o	← la console, à l'allumage — la mise en route de la console
main()	39 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre — attend de pouvoir écrire une case
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles — 10 lettres sur 43

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 3    32 o sur place, dans main() + EcrireNombre (33 o)
image()  × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <nombre> : écrire un nombre ----

#include <nombre> // nombre() : une valeur, en chiffres
```

2. UNE VARIABLE — SCORE

```
uint8_t score = 42; // une variable : sa valeur se lit pendant le jeu
```

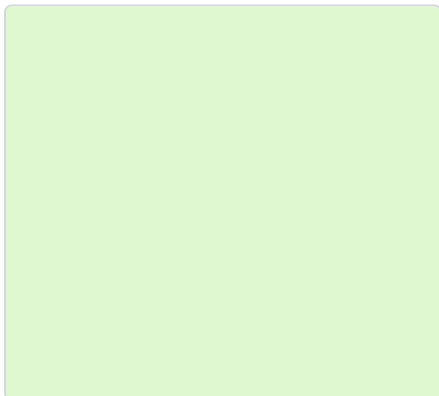
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    nombre(4, 8, score); // 042 : trois chiffres, des zéros devant
    nombre(4, 10, score, 2); // 42 : deux chiffres demandés
    nombre(4, 12, score + 8); // un calcul aussi : 050

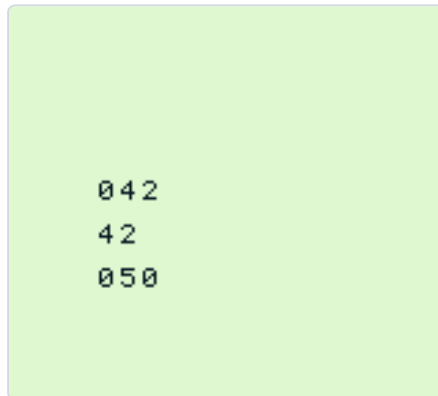
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

042, puis 42, puis 050, les uns sous les autres.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 042 en (4, 8) (« 042 »)

✓ 42 sur deux chiffres

✓ le calcul : 050

Ensuite : **0.8.4. #include <reste> — a % b** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++